

The cloud side — NETPIE as data and model backbone

How little the device can say, and what a subscriber with history can see

- The message schema: every field, what it is for, and what it costs in bytes
- Topics, the device shadow, retained messages — and which is for what
- Five publish policies, measured: bytes per hour against detection delay
- Model-based alerts against a fixed dashboard threshold, on a healthy load change
- Retrieving the history, and what the cloud sees that the device cannot
- The retraining loop, and the four checks that stop it making things worse

Notebook: `iaaiot26_Lect12_nb.ipynb`

What actually goes on the wire

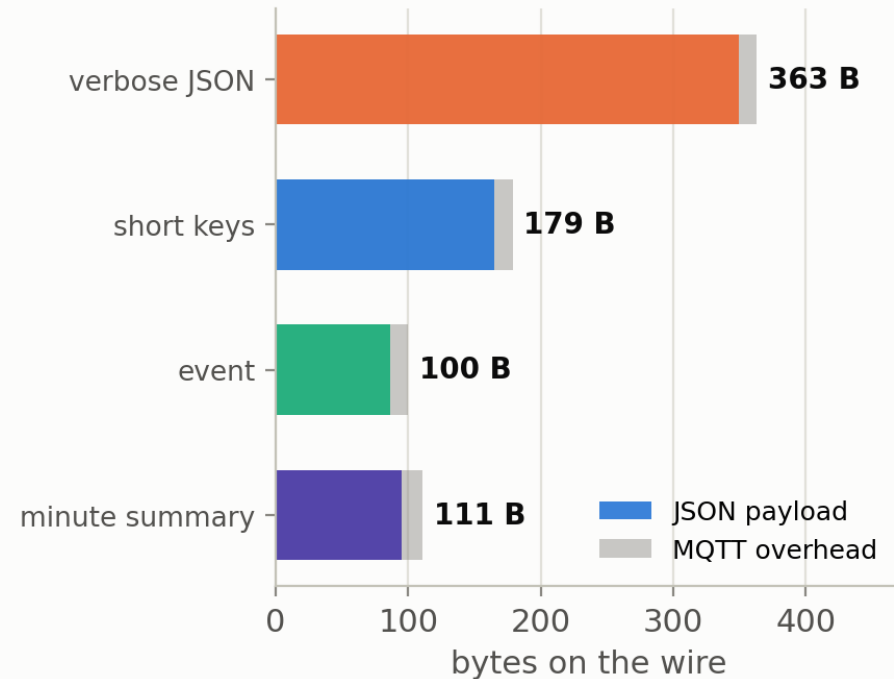
- A schema is a contract: version it, timestamp at the source, round to what you measured, and send the verdict as well as the evidence.

one window, on the wire

"v"	schema version	1
"t"	unix time, ms	1789000060.096
"f"	the twelve features	[0.0899, 0.320, ...]
"l"	the device's label	"normal"
"m"	classifier margin	1.47
"a"	anomaly score	2.61
"d"	which feature drove it	"kurt"
"r"	alarm raised	1

the last three fields are what Lecture 11 built, and the reason the cloud can tell a spill from a heavy cut

what each message costs

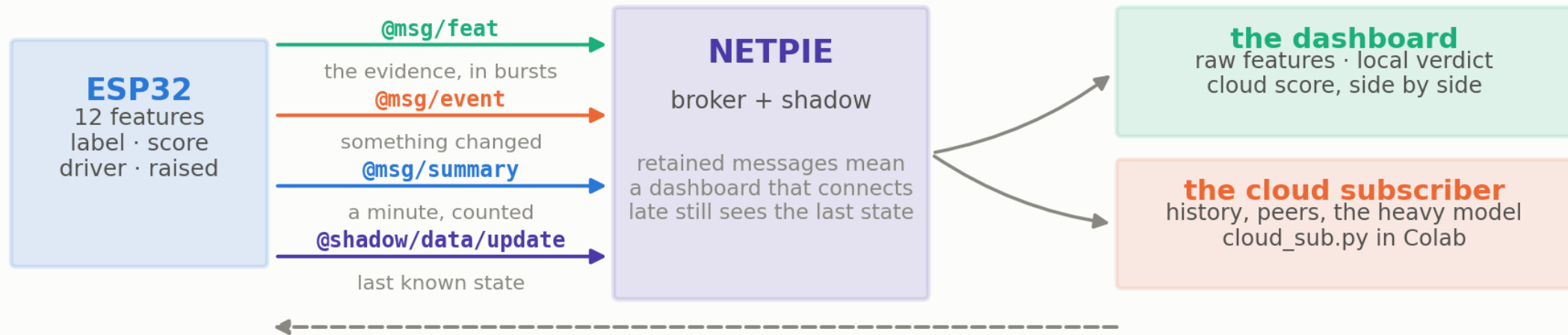


Self-documenting keys cost 184 bytes a message. At 15.6 windows a second that is 248 MB a day, for readability nobody reads.

The last three fields — score, driver, raised — are Lecture 11's output. They cost about thirty bytes, and without them every cheap policy on slide 5 has nothing to send.

Topics, the shadow, and retained messages

- A message is an event in a stream; the shadow is a current value — and confusing them is how a dashboard replays an hour to draw one tile.

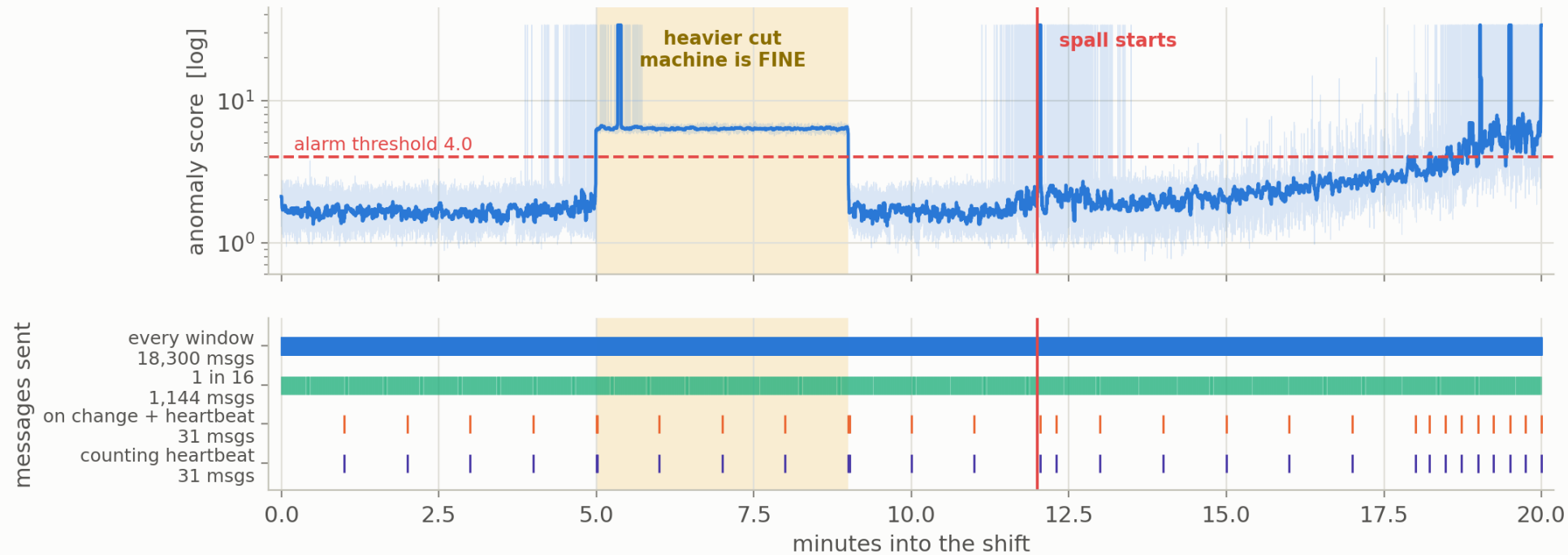


@private/model — new thresholds and coefficients, coming down. That arrow is Lecture 13.

One subscriber takes +/@msg/# from every machine in the plant: "+" matches the device id, "#" matches everything below it, and nothing has to be duplicated inside the payload.

One shift, and two things happening in it

- Twenty minutes of one machine, 18,300 windows. A heavier cut at minute 5 — the machine is fine — and a bearing spall from minute 12.

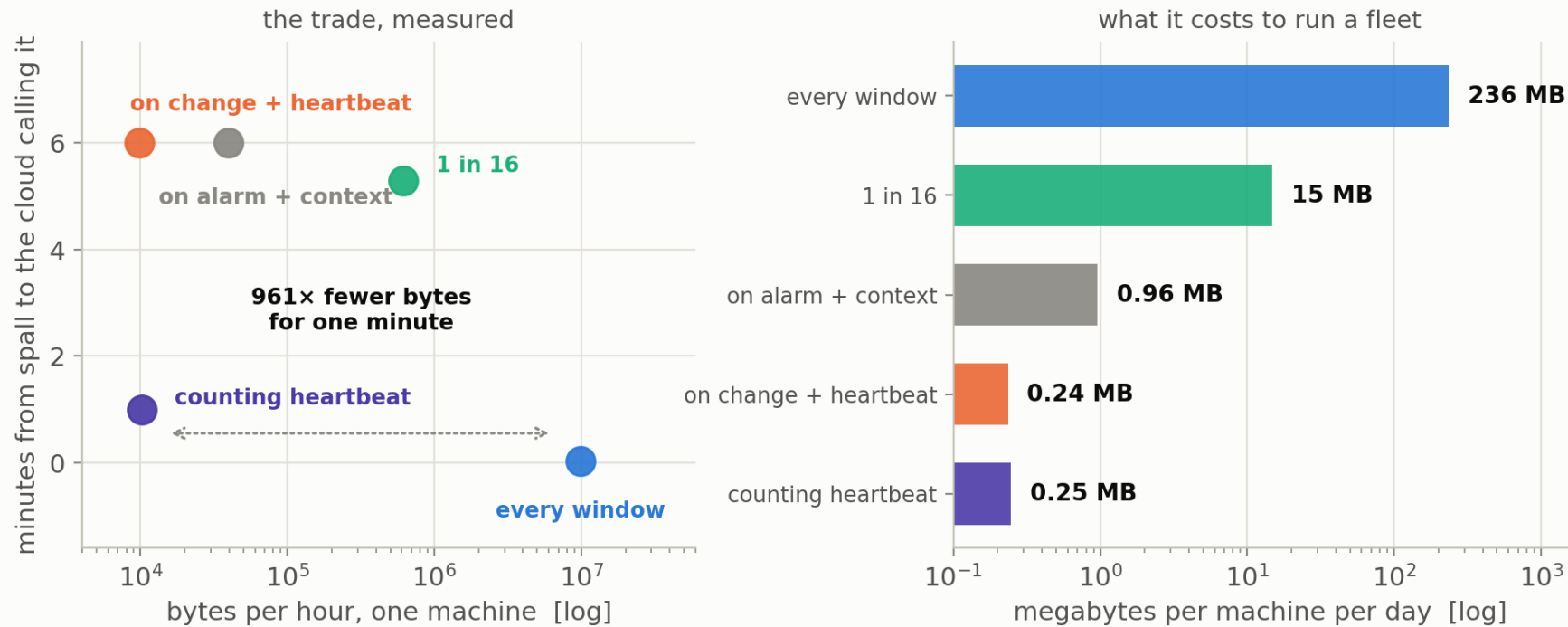


One machine, twenty minutes. Every tick is a message. The load change and the spall are both visible to the device — only one of them is worth anybody's attention.

The load change raises the alarm on 100 % of its windows, driven by rms. The spall raises it far less often, driven by kurt. The alarm alone cannot tell them apart; the alarm and the driver name can.

Bytes against delay

- Five policies, one cloud rule applied to all of them: alert on three messages in two minutes reporting an alarm with an impulsive driver.

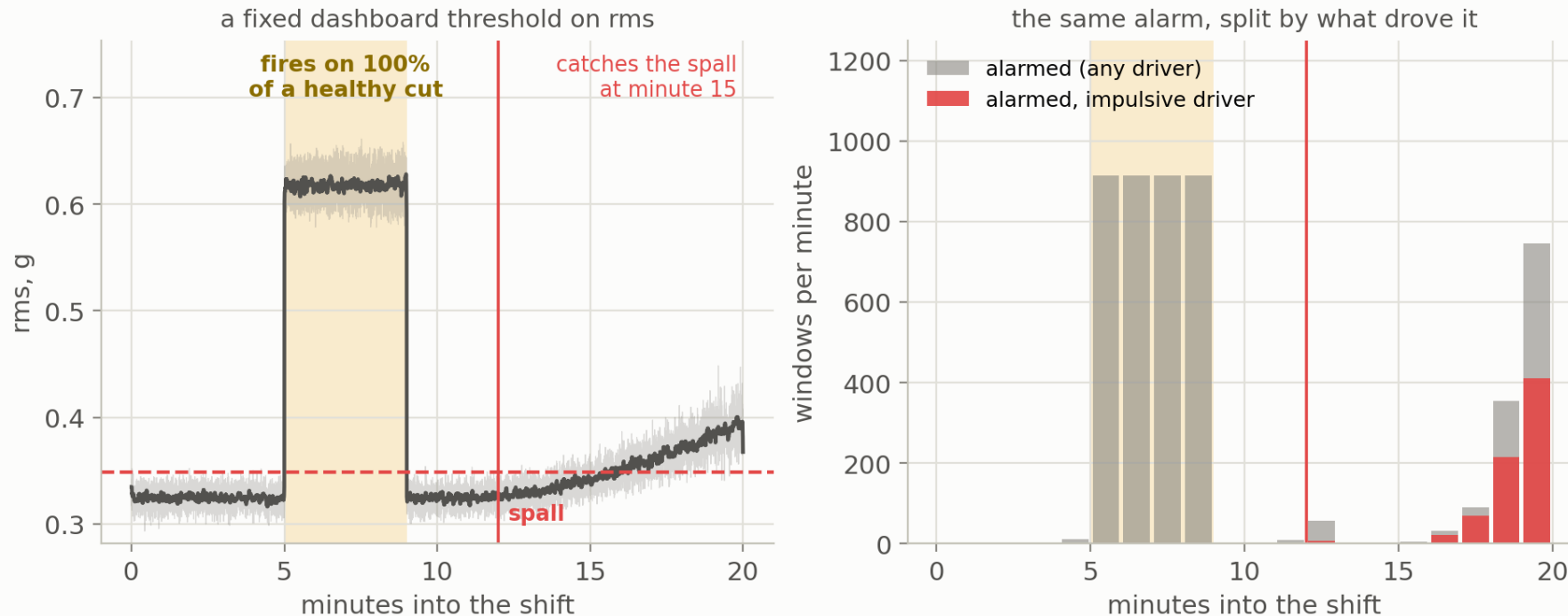


The counting heartbeat sends the same 31 messages as on-change and adds one integer: how many windows alarmed impulsively. Six bytes buy back five minutes.

Lecture 11: features go up, not waveforms. Lecture 12: STATISTICS go up, not samples — and the latency of a counting heartbeat is one heartbeat, whatever the window rate.

A fixed threshold, on a healthy machine

- NETPIE gives you “alert when rms exceeds 0.35” in a minute. On this shift it fires on every window of a healthy heavy cut.

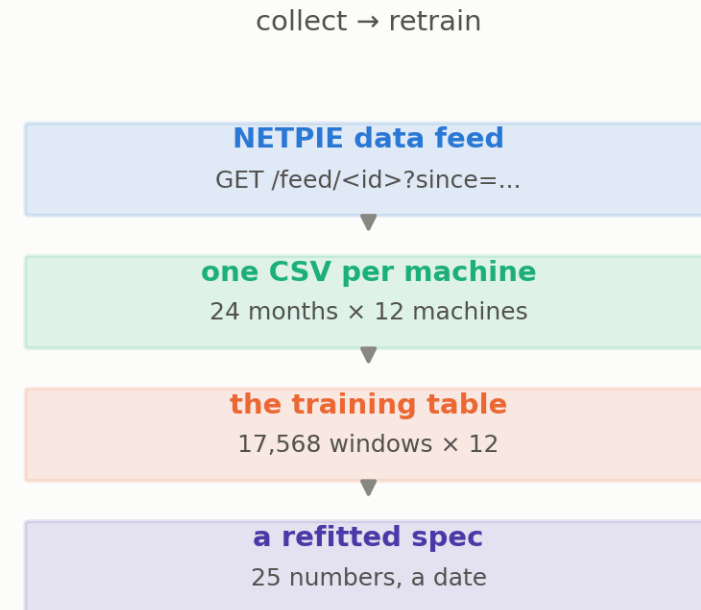
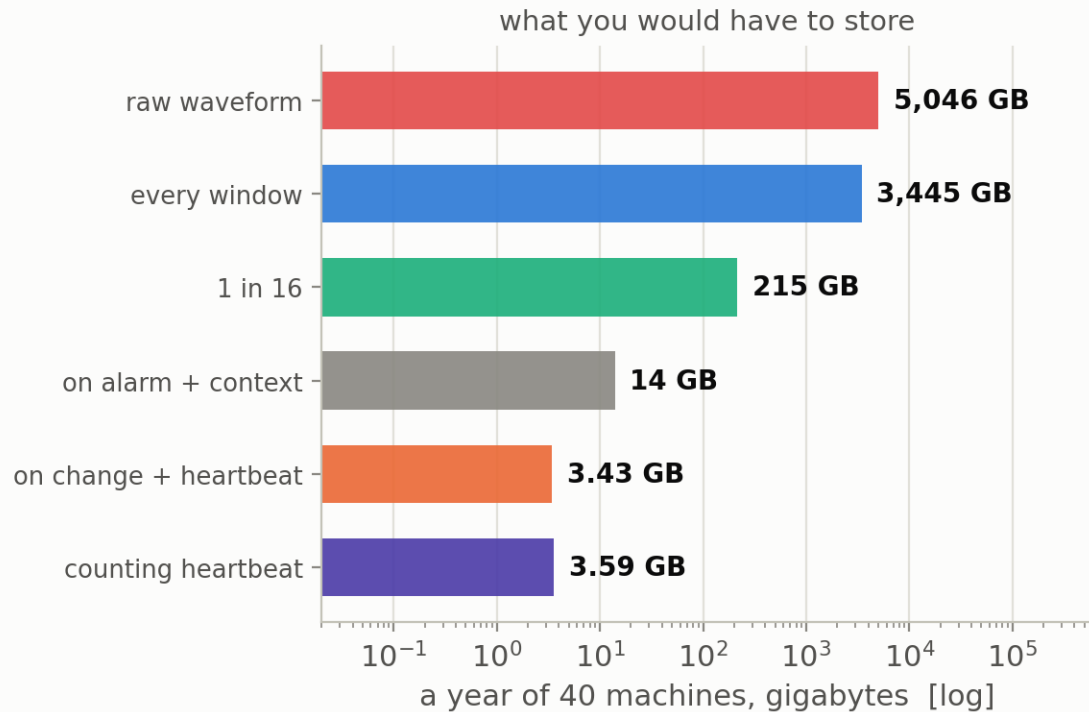


The load change and the spall both raise the alarm. One field — which feature drove it — separates them, and it is the field a fixed threshold does not have.

A threshold on one raw quantity carries no information about WHY that quantity moved — and "why" is what an alert is for. Show the driver first, the score second.

Keeping the history

- A year of 40 machines, under each policy. The publish policy is also a decision about what the next model will be allowed to learn from.

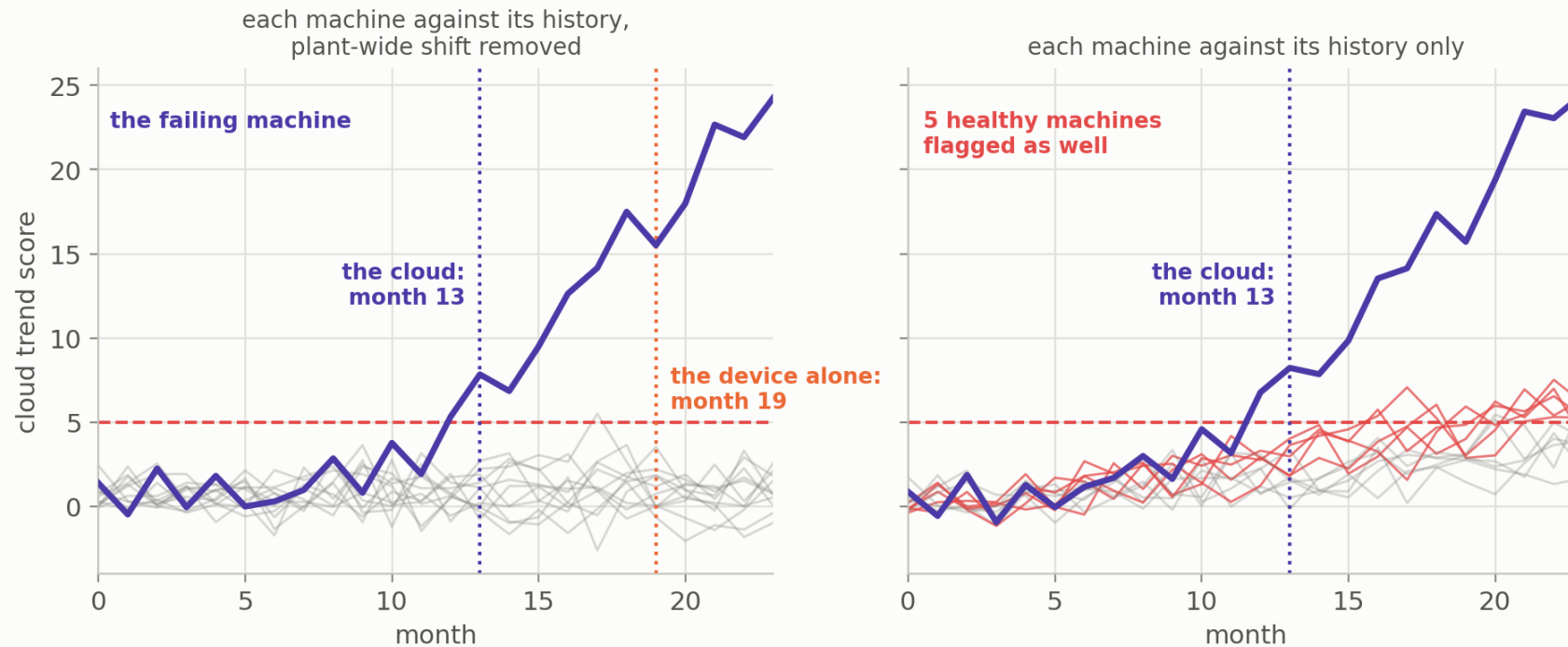


You cannot retrain on data you decided not to keep. The publish policy is also a decision about what the next model will be allowed to learn from.

That consequence arrives a year later, when nobody remembers making it. Decide what you will need to retrain on before you decide what to publish — not after.

What the cloud sees that the device cannot

- 12 machines, 24 months. One is failing slowly; meanwhile the whole plant's ambient level is rising, and nothing is wrong with that.

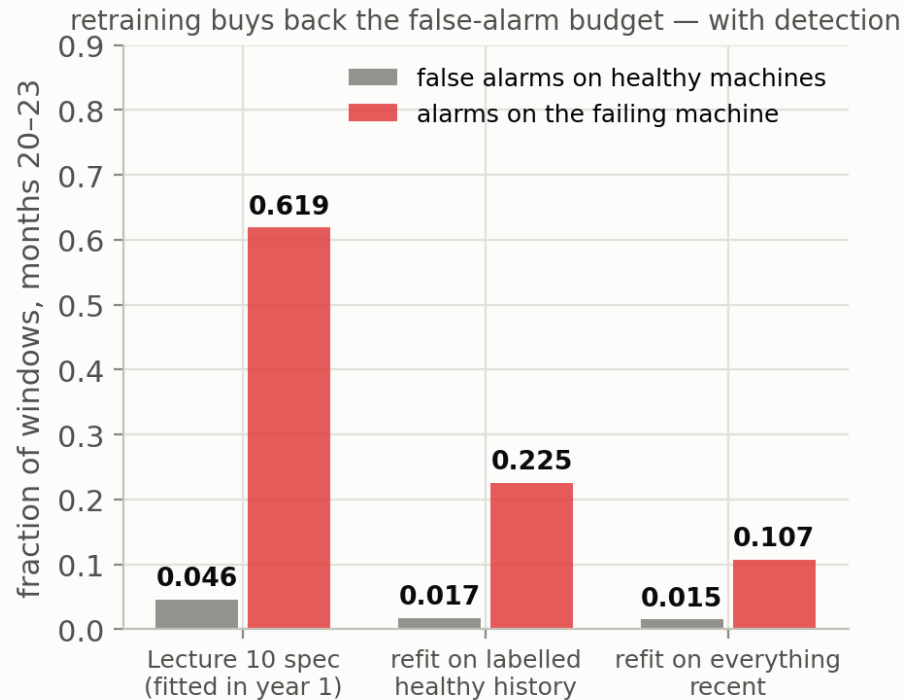


*6 months of warning, at half the spall severity — and only because the plant-wide change was removed first.
The same trend without the fleet reference condemns 5 healthy machines.*

6 months of warning, at half the spall severity — and the same trend WITHOUT the fleet reference condemns 5 healthy machines. The advantage is context, not a bigger model.

Retraining is not free

- Lecture 10 wrote "review after 12 months" into the spec. Two years on, its false-alarm rate has gone from 0.025 to 0.046.



the four gates on a retrain

1 • exclude what alarmed

a baseline that absorbs the fault
learns to accept it

2 • hold out the future

score the new spec on months it
has never seen

3 • re-measure detection

a spec that only improved its
false-alarm rate has not improved

4 • write the date on it

the spec expires; say when

Refitting on everything recent looks best on false alarms (0.015) and is the worst spec here: it has learned that the failing machine is normal (0.107).

Refitting on everything recent gives the best false-alarm rate on the slide (0.015) and the worst detector (0.107): it has learned that the failing machine is normal.

The cloud subscriber, in one loop

- Subscribe, keep everything, count the evidence, and publish a verdict the device does not depend on.

```
def on_message(client, userdata, msg):
    body = json.loads(msg.payload.decode())
    append_csv(body)                # section 7 retrains on this

    # one window is noise; ni is a whole minute of them
    k = max(1 if body.get('r') and body.get('d') in IMPULSIVE else 0,
            int(body.get('ni', 0)))
    now = body.get('t', time.time())
    for _ in range(k):
        _hits.append(now)
    while _hits and _hits[0] < now - ALERT_WINDOW:
        _hits.popleft()            # a rolling two-minute window

    if len(_hits) >= ALERT_K and not _alerted:
        client.publish(PUB_TOPIC, json.dumps(
            {'v': 1, 't': now, 'alert': 'bearing',
             'evidence': len(_hits), 'driver': body.get('d')}))
```

It never decides anything the machine depends on. The device drives its own actuator; this is a second opinion with a longer memory and a name on it.

Lab 12 — the uplink, end to end

A · publish

Add net_pub.py to your Lecture 11 device.
Connect to NETPIE, publish events,
heartbeats and shadow updates.

Verify with mosquitto_sub or the NETPIE
console that the messages arrive.

B · subscribe

Run cloud_sub.py in Colab. Watch the
evidence counter, and make it raise an alert
by switching WAVE to wave_bearing.

Measure your own bytes per hour for two
policies.

C · the dashboard

Build a NETPIE dashboard: raw features, the
local verdict, and the cloud alert side by side.

Then add the alert rule you would actually
defend, and say why it is not a threshold on
rms.

```
$ mosquitto_sub -h broker.netpie.io -p 1883 \  
    -u <token> -P <secret> -i <device-id> -t '@msg/#' -v  
  
@msg/summary {"v":1,"t":1789000060.1,"n":938,"a_med":1.66,  
              "a_max":3.77,"r":0,"ni":0,"l":"normal","d":"mean"}  
@msg/event   {"v":1,"t":1789001094.3,"e":"raise","l":"bearing",  
              "m":2.31,"a":4.08,"d":"kurt","r":1}
```

Deliverable: your measured bytes per hour for two policies, a screenshot of the dashboard, and the alert rule you would defend to a maintenance manager.

Statistics go up, not samples



1. Write the schema first, version it, timestamp at the source — and measure a message on the wire, not in memory.
2. Send the verdict as well as the evidence. The driver field costs about eight bytes and separates a heavy cut from a spall.
3. Never publish per window. On change plus a heartbeat is 1004× cheaper and detects sooner than decimation.
4. Put a statistic in the heartbeat. Six bytes took detection from 6.0 min to 1.0, at 961× fewer bytes than sending everything.
5. The cloud's advantage is context, not model size: history and peers were worth 6 months, and the fleet reference is what made them usable.
6. Gate every retrain. "Everything recent" scored best on false alarms (0.015) and had learned that the failing machine was normal.